

STAGE 4

STAGE 4: AUTOSAR ECU SW 아키텍처

AUTOSAR Classic/Adaptive/Foundation, SOME/IP, SecOC, Eclipse Velocitas

주요 참고 기관 및 표준

- AUTOSAR (CP/AP/FO R24-11)
- COVESA (vSOME/IP, VSS)
- MathWorks (Simulink AUTOSAR Blockset)
- Elektrobit (EB tresos)
- ETAS (RTA-OS)
- Vector Informatik (DaVinci, MICROSAR)
- Lauterbach
- Eclipse Foundation (Velocitas)

본 문서는 자동차 소프트웨어 엔지니어를 위한 한국어 심화 교육 자료입니다. 국제 표준, 정부 규제, 업계 가이드라인을 종합하여 실무 중심으로 재구성했습니다.

STAGE 4. AUTOSAR ECU 소프트웨어 아키텍처

AUTOSAR(AUTomotive Open System ARchitecture)는 전 세계 자동차 OEM·Tier1·반도체·소프트웨어 기업이 공동으로 개발하는 개방형 표준화 소프트웨어 플랫폼입니다. ECU 소프트웨어의 이식성, 재사용성, 상호운용성을 보장하며, 현대 차량 ECU의 사실상 표준 아키텍처입니다.



그림: AUTOSAR Classic Platform 계층 아키텍처

[그림] AUTOSAR Classic Platform 계층 구조 — BSW·RTE·SWC

1. AUTOSAR 플랫폼 로드맵

플랫폼	약칭	목표	주요 특징	최신 릴리즈
Classic Platform	CP	마이크로컨트롤러 기반 임베디드 ECU	OSEK 기반 RTOS, 정적 구성, 안전 임계 시스템	R24-11
Adaptive Platform	AP	고성능 컴퓨팅 플랫폼 (HPC, 자율주행)	POSIX OS, 동적 서비스 배포, SOME/IP	R24-11
Foundation	FO	CP/AP 공통 표준 (통신, 보안 등)	SOME/IP, 보안 요구사항, 방법론	R24-11

2. AUTOSAR Classic Platform (CP) 아키텍처

AUTOSAR CP는 3계층 구조(SWC ↔ RTE ↔ BSW)로 구성됩니다. 이 계층 분리로 SW 컴포넌트(SWC)는 하드웨어에 독립적으로 개발·이식될 수 있습니다.

2.1 SW 컴포넌트 계층 (SWC)

SW 컴포넌트(Software Component)는 기능 로직을 구현하는 AUTOSAR의 기본 단위입니다. 포트(Port)와 인터페이스(Interface)를 통해서만 외부와 통신합니다.

SWC 유형	설명	예시
Application SWC	응용 기능 로직 구현	PID 제어기, 경고 로직
Sensor/Actuator SWC	센서·액추에이터 직접 접근 (I/O Abstraction을 통해)	온도 센서 드라이버 래퍼
ECU-Abstraction SWC	ECU 하드웨어 추상화	LED 제어 추상화
Complex Device Driver (CDD)	BSW 미지원 하드웨어 직접 제어	독자 통신 모듈
Composition SWC	여러 SWC를 그룹화	도어 제어 컴포지션
Service SWC	BSW 서비스를 SWC 형태로 노출	진단 이벤트 관리자

2.2 RTE (Runtime Environment)

RTE는 AUTOSAR CP의 통신 미들웨어입니다. SWC 간 통신과 SWC-BSW 간 통신을 추상화합니다. 같은 ECU 내 SWC 간은 "로컬 통신(Sender-Receiver, Client-Server)", 다른 ECU의 SWC 간은 "ECU 간 통신(COM을 통해 CAN/Ethernet 전송)"으로 처리됩니다.

통신 패턴	설명	사용 시점
Sender-Receiver	비동기 데이터 전달, 큐 또는 최신값 방식	센서 값 배포, 이벤트 전파
Client-Server	요청-응답 패턴, 동기/비동기	서비스 호출, 진단 기능 요청
Mode Switch	시스템 모드 변화 알림	ECU 파워 모드 전환
Trigger	이벤트 트리거 인터페이스	외부 인터럽트 기반 연산

2.3 BSW — 기본 소프트웨어 계층

BSW(Basic Software)는 OS, 통신, 메모리, 진단 등 공통 인프라 서비스를 제공하는 AUTOSAR CP의 기반 계층입니다. 4개 서브계층으로 구성됩니다.

BSW 서브계층	주요 모듈	기능
서비스 계층	OS, WDG, DEM, DCM, NvM, COM	OS 스케줄링, 진단, 메모리, 통신
ECU 추상화 계층	MCAL 추상화 모듈들 (IoHwAb)	MCU 독립 하드웨어 추상화
마이크로컨트롤러 추상화 (MCAL)	ADC, DIO, PWM, SPI, CAN, LIN drv	레지스터 직접 접근 최하위 드라이버
복합 드라이버	Complex Device Drivers	AUTOSAR 표준 범위 외 하드웨어

2.4 주요 BSW 모듈 상세

모듈	약칭	기능
OS (Operating System)	OS	OSEK/VDX 기반 RTOS, 태스크·ISR·알람 관리
Watchdog Manager	WDG/WDGM	SW 데드라인 감시, 생존 감지
Diagnostic Communication Manager	DCM	UDS 서비스 처리 (0x10~0x87)
Diagnostic Event Manager	DEM	DTC 저장·관리, 동결 프레임

모듈	약칭	기능
NV Data Manager	NvM	비휘발성 메모리 읽기/쓰기 추상화
Communication Manager	ComM	통신 채널 네트워크 관리
PDU Router	PduR	PDU 라우팅 (CAN→COM, COM→CAN FD 등)
CAN Interface	CanIf	CAN 드라이버 추상화, PDU 매핑
CAN State Manager	CanSM	CAN 버스 상태 관리
LIN Interface	LinIf	LIN 드라이버 추상화, 스케줄 테이블
Ethernet Interface	EthIf	Ethernet 드라이버 추상화
Socket Adaptor	SoAd	TCP/UDP 소켓 ↔ PDU 변환
Secure On-board Communication	SecOC	CAN/Eth 메시지 MAC 인증
Flash Boot Loader	FBL	플래시 프로그래밍 지원 부트로더
E2E Protection Library	E2E	통신 데이터 무결성 보호 (CRC+Counter)

3. AUTOSAR OS (운영체제)

AUTOSAR CP의 OS는 OSEK/VDX OS를 기반으로 확장됩니다. 정적 구성(빌드 타임 결정), 선점형 스케줄링, 태스크·ISR·알람·스케줄 테이블을 지원합니다.

개념	설명
Task	기본 실행 단위, 우선순위·활성화 조건 정의
ISR (Interrupt Service Routine)	인터럽트 핸들러 Category 1/2
Alarm	카운터 기반 주기 태스크 활성화
Schedule Table	복잡한 타이밍 시퀀스 정의 (자동차 ECU 주기 관리)
Resource	공유 자원 보호 (Priority Ceiling Protocol)
Event	태스크 간 동기화 메커니즘

주요 AUTOSAR OS 구현체: ETAS RTA-OS, EB tresos Studio, Vector MICROSAR OS, KPIT KPC OS. Lauterbach TRACE32는 AUTOSAR OS 인식 디버거로 태스크·ISR 실행 추적이 가능합니다.

4. SOME/IP — Service-Oriented Middleware over IP

SOME/IP(Scalable service-Oriented MiddleWare over IP)는 AUTOSAR Foundation에서 표준화된 자동차 이더넷 기반 미들웨어 프로토콜입니다. AUTOSAR AP 아키텍처에서 서비스 지향 통신을 구현합니다.

기능	설명
서비스 디스커버리 (SOME/IP-SD)	UDP 멀티캐스트로 서비스 제공자·소비자 자동 발견
원격 프로시저 호출	SOME/IP RPC — 요청-응답 방식 서비스 호출
이벤트 알림 (Event)	구독-발행 패턴으로 이벤트 전달
필드 (Field)	게터·세터·알림이 결합된 서비스 요소
직렬화	SOME/IP PDU 직렬화 — 빅엔디안, 가변 길이

vsomeip(COVESA GitHub)는 SOME/IP의 오픈소스 C++ 구현체입니다. Eclipse Velocitas SDK도 SOME/IP 기반 차량 앱 개발을 지원합니다.

5. AUTOSAR Adaptive Platform (AP)

AUTOSAR AP는 자율주행·인포테인먼트 등 고성능 컴퓨팅이 필요한 도메인을 위한 플랫폼입니다. Linux/QNX 기반 POSIX OS 위에서 동작하며, 동적 서비스 배포와 OTA 업데이트를 지원합니다.

ara:: 네임스페이스	기능 클러스터	역할
ara::com	Communication Management	SOME/IP 바인딩, 서비스 인터페이스 추상화
ara::exec	Execution Management	Adaptive Application 수명 주기 관리
ara::phm	Platform Health Management	응용 프로그램 감시, 복구 동작
ara::diag	Diagnostic Management	UDS/SOVD 진단 처리
ara::crypto	Cryptography	암호화, 키 관리, 인증서
ara::iam	Identity & Access Management	서비스 접근 제어, 권한 관리
ara::per	Persistency	KVS, 파일 기반 영구 저장
ara::log	Log & Trace	분산 로깅, DLT(Diagnostic Log and Trace)

ara:: 네임스페이스	기능 클러스터	역할
ara::tsync	Time Synchronization	IEEE 802.1AS 기반 시간 동기화
ara::nm	Network Management	네트워크 모드 관리

5.1 CP vs AP 비교

비교 항목	CP (Classic Platform)	AP (Adaptive Platform)
대상 하드웨어	MCU (Infineon AURIX, NXP S32K)	MPU/SoC (NXP S32G, Renesas R-Car)
운영체제	AUTOSAR OS (OSEK 기반)	POSIX OS (Linux, QNX, INTEGRITY)
통신	CAN, LIN, FlexRay + COM/PDU	Automotive Ethernet + SOME/IP
배포	빌드 타임 정적 구성	런타임 동적 서비스 배포
기능안전	ASIL D 지원	ASIL B(현재), ASIL D 목표
주요 적용	엔진, 변속기, ABS, 에어백	자율주행 HPC, 중앙 게이트웨이
개발 언어	C (MISRA-C)	C++14/17, Ada

6. AUTOSAR 개발 워크플로우

AUTOSAR CP 개발은 모델 기반 설계(MBD) 방법론을 따릅니다. arxml(AUTOSAR XML) 파일이 전체 구성의 핵심입니다.

- ▶ 1. 아키텍처 설계: SWC·포트·인터페이스 정의 (DaVinci Architect, Simulink AUTOSAR Blockset)
- ▶ 2. BSW 구성: OS·통신·진단 모듈 파라미터 설정 (DaVinci Configurator, EB tresos Studio)
- ▶ 3. 코드 생성: arxml에서 RTE·BSW 코드 자동 생성 (DaVinci Developer, arxtract)
- ▶ 4. 응용 코드 작성: SWC Runnable 구현 (C 코드, Simulink 모델)
- ▶ 5. 빌드·통합: AUTOSAR 전체 BSW+SWC 빌드 (GCC, IAR, Green Hills)
- ▶ 6. 검증: SIL(소프트웨어 IN-THE-LOOP), PIL, HIL 테스트

7. AUTOSAR 보안 — SecOC & Crypto 스택

AUTOSAR CP/AP 모두 보안 기능을 표준 모듈로 포함합니다. CP에서는 HSM(Hardware Security Module)과 연동하여 AES-CMAC 기반 SecOC를 구현합니다.

보안 기능	모듈	설명
메시지 인증	SecOC + CSM	CAN/Eth 메시지 MAC 생성·검증
보안 부트	FBL + HSM	펌웨어 서명 검증, 부트 체인 보호
OTA 무결성	Crypto Stack	업데이트 패키지 서명 검증
키 관리	KeyM	HSM 내 키 프로비저닝, 갱신
암호화	CSM (Crypto Service Manager)	AES, RSA, SHA, ECDSA 추상화 API

8. Eclipse Velocitas — 차량 앱 개발 SDK

Eclipse Velocitas(Eclipse Foundation)는 AUTOSAR AP-COVESA VSS 기반 차량 애플리케이션(Vehicle App) 개발 SDK입니다. Python/C++ SDK, 컨테이너 기반 빌드, GitHub Actions CI/CD, KUKSA.val 데이터 브로커 연동을 지원합니다.

Velocitas 사용 예: Vehicle Speed 구독 → 100km/h 초과 시 알림. VSS 신호(Vehicle.Speed)를 SOME/IP 없이 KUKSA val 데이터 브로커로 접근합니다.

9. SOAFEE — 클라우드 네이티브 임베디드

SOAFEE(Scalable Open Architecture for Embedded Edge)는 Arm 주도 컨소시엄이 개발한 클라우드 네이티브 기술을 차량 임베디드 시스템에 적용하는 아키텍처 사양입니다. 컨테이너(Docker/OCI), 오케스트레이션(Kubernetes 기반 경량 k3s), CI/CD를 차량 ECU에 적용합니다.

10. AUTOSAR arxml 파일 구조 심화

arxml(AUTOSAR XML)은 AUTOSAR 설계 정보를 교환하는 표준 XML 형식입니다. SWC 정의, 포트 인터페이스, 데이터 타입, 시스템 구성 등 모든 설계 정보가 arxml에 담깁니다. DaVinci Developer(Vector), EB tresos Studio(Elektrobit), ISOLAR-A(ETAS)가 arxml 편집기입니다.

arxml 요소	역할	예시
APPLICATION-SW-COMPONENT-TYPE	Application SWC 정의	SpeedController
PORT-INTERFACE	포트 인터페이스 정의 (SR/CS)	
DATA-ELEMENT	포트에 흐르는 데이터 정의	VehicleSpeed
P-PORT-PROTOTYPE	제공(Provide) 포트	SWC가 데이터/서비스를 제공
R-PORT-PROTOTYPE	요구(Require) 포트	SWC가 데이터/서비스를 소비
RUNNABLE-ENTITY	실행 가능한 코드 단위	10ms 주기 TaskInit, Task10ms
INTERNAL-BEHAVIOR	SWC 내부 동작 정의	포트 접근, Runnable 스케줄링

arxml 요소	역할	예시
SYSTEM	전체 ECU 네트워크 시스템	ECU, 버스, 신호 전체 구성

arxml 개발 워크플로우: ① AUTOSAR 설계 도구에서 SWC/Interface 정의 → ② RTE 생성기가 arxml을 읽어 Rte.c/Rte.h 생성 → ③ BSW 설정 도구에서 BSW 모듈 설정(arxml 업데이트) → ④ BSW 코드 생성 → ⑤ 컴파일러로 전체 빌드 → ⑥ JTAG 디버거로 타겟 배포·검증.

11. RTE 생성 코드 패턴 — Sender/Receiver & Client/Server

RTE(Runtime Environment)가 생성하는 API는 SWC 코드에서 호출하여 포트를 통해 통신합니다. AUTOSAR에서 SWC는 RTE API만 사용하며, 직접 하드웨어나 다른 SWC를 호출하지 않습니다. 이 원칙이 SWC의 이식성과 재사용성을 보장합니다.

통신 패턴	RTE API	설명
SR 쓰기 (Provide)	Rte_Write__(value)	데이터 버퍼에 값을 씀
SR 읽기 (Require)	Rte_Read__(&value;)	데이터 버퍼에서 값을 읽음
SR 수신 상태	Rte_Receive__(&value;)	이벤트 큐에서 읽기 (Data Queue)
CS 호출 (동기)	Rte_Call__(args)	서버 Runnable 즉시 호출, 결과 대기
CS 호출 (비동기)	Rte_Call__async(args)	서버 Runnable 호출 후 계속 진행
CS 결과 수신	Rte_Result__(args)	비동기 CS 호출 결과 수신
모드 읽기	Rte_Mode__()	현재 모드 값 읽기
인터 Runnable 변수	Rte_IrvRead/IrvWrite	같은 SWC의 Runnable 간 데이터 공유

```
SWC 코드 예시(속도 모니터): void SpeedMonitor_10ms(void) { uint16 speed;  
Rte_Read_Req_VehicleSpeed(&speed); if (speed > 200) {  
Rte_Write_Prov_SpeedWarning(TRUE); } }
```

12. BSW 모듈 설정 실무 가이드

BSW(Basic Software) 모듈 설정은 AUTOSAR 프로젝트에서 가장 많은 시간이 소요되는 작업 중 하나입니다. EB tresos Studio 또는 Vector DaVinci Configurator로 설정하며, 잘못된 설정은 런타임 오류를 유발합니다.

BSW 모듈	핵심 설정 항목	주의 사항
ComStack (PduR/CanIf/Com)	PDU 매핑, 버스 ID, 메시지 우선순위	ComTimeout, 신호 초기값 설정 필수
DCM (진단)	SID별 세션/보안 레벨 권한, DID 범위	SecurityAccess 알고리즘 외부 구현
DEM (이벤트 관리)	DTC 코드, 이벤트 카테고리, 고장 임계	이벤트 조합 조건(HiCombination) 설정
NvM (비휘발성 저장)	블록 ID, 크기, 체크섬 유형	WriteAll 호출 시점, 쓰기 주기 주의
OS (AUTOSAR OS)	Task 우선순위, 스택 크기, 알람 설정	스택 오버플로 방지, 데드라인 모니터링
WdgM (워치독 관리)	검사점(Checkpoint), 타임아웃	모든 안전 경로에 체크포인트 배치

13. SOME/IP 서비스 정의 & 구현 상세

SOME/IP(Scalable service-Oriented MiddlewarE over IP)는 AUTOSAR Foundation의 미들웨어 프로토콜로, 이더넷 기반 서비스 지향 통신을 제공합니다.

SOME/IP-SD(Service Discovery)가 서비스 제공자와 소비자를 자동으로 연결합니다.

SOME/IP 헤더 필드	크기	설명
Service ID	2 바이트	서비스 식별자 (0x0001~0xFFFE)
Method/Event ID	2 바이트	메소드(0x0001~0x7FFF) 또는 이벤트(0x8001~0xFFFE)
Length	4 바이트	헤더 이후 페이로드 길이
Client ID	2 바이트	클라이언트 인스턴스 식별자
Session ID	2 바이트	요청-응답 쌍 매핑용 시퀀스 번호
Protocol Version	1 바이트	현재 0x01
Interface Version	1 바이트	서비스 인터페이스 버전
Message Type	1 바이트	0x00=REQUEST, 0x80=NOTIFICATION, 0x02=RESPONSE

SOME/IP 헤더 필드	크기	설명
Return Code	1 바이트	0x00=OK, 0x01=UNKNOWN_SERVICE 등

SOME/IP 통신 패턴	설명	사용 사례
RPC (Remote Procedure Call)	Client→Server 요청, Server→Client 응답	액추에이터 제어, 데이터 조회
Event Notification	Server가 구독자에게 비동기 이벤트 전송	속도, 온도 등 주기 데이터
Field	초기값 + 이벤트 알림 + Getter/Setter	설정값, 상태 정보
Service Discovery	OfferService/FindService/Subscribe	런타임 서비스 자동 검색

14. AUTOSAR OS 설정 & 스케줄링

AUTOSAR OS는 OSEK/VDX OS 표준을 기반으로 하는 실시간 운영체제입니다. 정적 설정(컴파일 타임 결정), 우선순위 기반 선점형 스케줄링, 시간 보호(Time Protection)가 특징입니다.

AUTOSAR OS 개념	설명	설정 예시
Task	스케줄링 단위, Basic Task/Extended Task	Task_10ms: 우선순위 10, 스택 1KB
ISR (Category 2)	인터럽트 서비스 루틴 (OS 인식)	CAN 수신 ISR, ADC 완료 ISR
Alarm	특정 카운터 조건에서 태스크 활성화	10ms 주기 알람으로 Task_10ms 활성화
Schedule Table	복수의 Task를 정해진 시점에 활성화	Schedule_1ms: 0ms에 1ms 태스크, 5ms에 5ms 태스크
Resource	Priority Ceiling으로 우선순위 역전 방지	RES_SCHEDULER: 최고 우선순위 리소스
Event	태스크 동기화 (WaitEvent/SetEvent)	Task_20ms가 Task_5ms 완료를 기다림

AUTOSAR OS 스케줄링 전략: BCC1(Basic Task, 단일 활성화), BCC2(Basic Task, 다중 활성화), ECC1(Extended Task), ECC2(Extended Task+다중 활성화). ASIL D

시스템에서는 Stack/Time 보호, 메모리 보호(MPU)가 필수입니다. ETAS RTA-OS, EB tresos OS가 AUTOSAR 인증 OS를 제공합니다.

15. SecOC 구현 가이드 — 메시지 인증

SecOC(Secure Onboard Communication)는 AUTOSAR 표준으로 CAN/Ethernet 메시지에 MAC(Message Authentication Code)을 추가하여 인증합니다. UN R155 사이버보안 요구사항 충족에 핵심적입니다.

SecOC 처리 단계	송신측	수신측
1. Freshness Value 생성	Freshness Manager에서 카운터 값 제공	Freshness Manager에서 예상 카운터 확인
2. MAC 입력 데이터	PDU 데이터 + Freshness Value 조합	수신 PDU + 예상 Freshness 조합
3. MAC 계산	CMAC(AES-128) 또는 HMAC-SHA256	동일 알고리즘으로 재계산
4. MAC 절단(Truncation)	전체 MAC의 일부(예: 24~64 bit) 추출	수신 MAC과 비교
5. SecPDU 구성	PDU + Truncated MAC + Freshness 일부	검증: 불일치 시 PDU 폐기

Freshness 카운터 유형	특징	사용 사례
Trip Counter	시동 ON/OFF 기반 카운터	파워트레인 메시지

Freshness 카운터 유형	특징	사용 사례
Message Counter	메시지별 독립 카운터	안전 임계 메시지
Session Counter	진단 세션 기반 카운터	진단 메시지
Time-based	GPS 시간 동기화 기반	V2X 메시지

16. AUTOSAR CP 기능안전 (FuSa) 적용

AUTOSAR Classic Platform은 ISO 26262 파트 4~6에 따른 ASIL B~D 수준의 소프트웨어 개발을 지원합니다. 안전 관련 SWC는 별도 파티션(Partition)에서 실행되고, OS Memory Protection으로 격리됩니다.

AUTOSAR 안전 메커니즘	기능	ISO 26262 대응
WdgM (Watchdog Manager)	Runnable Supervision, Alive Supervision	Part 6: 소프트웨어 안전 메커니즘
E2E Protection	PDU 데이터 무결성 체크 (CRC+카운터+DataID)	Part 4: 시스템 안전 메커니즘
OS Memory Protection	SWC 메모리 영역 분리 (MPU)	Part 6: 공간 분리
Dem (DEM)	안전 관련 DTC 기록 및 상태 관리	Part 7: 운용 요구사항
FiM (Function Inhibition Manager)	결함 시 기능 비활성화	Part 4: 오류 처리
OS Time Protection	태스크 실행 시간 감시	Part 6: 시간 분리

E2E Protection 프로파일: Profile 1/1M(CRC8 + 4bit 카운터 + DataID), Profile 2(CRC8 + 4bit 카운터), Profile 4/4M(CRC32 + 16bit 카운터 + 32bit DataID). ASIL D에는 Profile 4M(32bit CRC, 무결성 최고)이 적합합니다. 적용 규칙: 수신측 E2E 오류 시

즉각 최소 위험 조건(MRC)으로 전환.

17. AUTOSAR 실무 틀체인 비교

AUTOSAR 프로젝트의 전체 개발 환경은 여러 도구의 통합으로 구성됩니다. 주요 도구 공급업체와 각 도구의 역할을 이해해야 프로젝트에서 효율적으로 협업할 수 있습니다.

단계	Vector 도구	ETAS (Bosch) 도구	Elektrobit (EB) 도구
아키텍처 설계	DaVinci Developer	ISOLAR-A (모델링)	EB tresos Studio
BSW 설정	DaVinci Configurator	EB tresos Studio	EB tresos Studio
BSW 코드 생성	MICROSAR (BSW 라이브러리)	RTA-BSW	EB tresos (BSW)
RTE 생성	DaVinci Developer (RTE 생성)	ISOLAR (RTE 생성)	EB tresos (RTE)
OS	MICROSAR OS	RTA-OS (ETAS)	EB tresos OS
모델 기반 개발	Simulink AUTOSAR Blockset	ASCET (ETAS)	Simulink 연동
디버깅	CANoe (진단), Lauterbach	LABCAR (HIL)	Lauterbach, IAR

OEM 선호도: BMW 그룹은 EB tresos, VW/Audi는 Vector MICROSAR, Toyota는 내부 도구 또는 Vector. 한국 현대/기아는 Vector MICROSAR + DaVinci 조합이 주류입니다. 공급업체 선택은 OEM과의 협의 및 기존 공급망 호환성에 달려 있습니다.

학습 체크리스트

아래 항목을 모두 이해하고 설명할 수 있으면 다음 단계로 진행하세요.

AUTOSAR CP와 AP의 목적과 차이를 설명할 수 있다

AUTOSAR CP의 3계층 구조(SWC, RTE, BSW)를 그림으로 설명할 수 있다

6가지 SWC 유형과 각각의 목적을 설명할 수 있다

RTE의 Sender-Receiver와 Client-Server 통신 패턴 차이를 설명할 수 있다

BSW의 4개 서브계층과 주요 모듈(DCM, DEM, NvM, ComM)을 설명할 수 있다

AUTOSAR OS의 Task·Alarm·Schedule Table 개념을 설명할 수 있다

SOME/IP의 서비스 디스커버리(SOME/IP-SD)를 설명할 수 있다

SOME/IP의 RPC·이벤트·필드 통신 패턴을 설명할 수 있다

AUTOSAR AP의 10개 ara:: 기능 클러스터를 나열하고 설명할 수 있다

AUTOSAR CP 개발 워크플로우(아키텍처→BSW구성→코드생성→검증)를 설명할 수 있다

SecOC가 어떻게 CAN 메시지를 보호하는지 설명할 수 있다

E2E Protection Library의 목적을 설명할 수 있다

Eclipse Velocitas의 역할과 구성 요소를 설명할 수 있다

SOAFEE가 자동차에 클라우드 네이티브를 어떻게 적용하는지 설명할 수 있다

arxml 파일이 AUTOSAR 개발에서 어떤 역할을 하는지 설명할 수 있다

한/영 용어집

한국어 용어	영문 용어	설명
개방형 시스템 아키텍처	AUTOSAR (AUTomotive Open System ARchitecture)	자동차 SW 표준화 플랫폼 컨소시엄 및 표준
클래식 플랫폼	CP (Classic Platform)	MCU 기반 정적 AUTOSAR 플랫폼
어댑티브 플랫폼	AP (Adaptive Platform)	POSIX OS 기반 동적 AUTOSAR 플랫폼 (HPC용)
SW 컴포넌트	SWC (Software Component)	AUTOSAR 기능 로직의 기본 캡슐화 단위
런타임 환경	RTE (Runtime Environment)	SWC 간, SWC-BSW 간 통신을 추상화하는 미들웨어
기본 소프트웨어	BSW (Basic Software)	OS·통신·메모리·진단 공통 서비스 계층
마이크로컨트롤러 추상화 계층	MCAL (Microcontroller Abstraction Layer)	레지스터 직접 접근 하드웨어 드라이버 계층
포트 인터페이스	Port Interface	SWC 간 데이터·서비스 교환 인터페이스 정의
실행 가능 엔티티	Runnable	RTE가 호출하는 SWC 내 함수 단위

한국어 용어	영문 용어	설명
SOME/IP	SOME/IP (Scalable service-Oriented MiddleWare over IP)	AUTOSAR AP/FO 이더넷 미들웨어 프로토콜
서비스 디스커버리	SOME/IP-SD (Service Discovery)	SOME/IP 서비스 자동 검색 프로토콜
진단 통신 관리자	DCM (Diagnostic Communication Manager)	UDS 진단 서비스를 처리하는 BSW 모듈
진단 이벤트 관리자	DEM (Diagnostic Event Manager)	DTC 저장·상태 관리 BSW 모듈
비휘발성 메모리 관리자	NvM (NV Memory Manager)	EEPROM/Flash 읽기·쓰기 추상화 BSW 모듈
통신 관리자	ComM (Communication Manager)	네트워크 통신 채널 상태 관리 BSW 모듈
PDU 라우터	PduR (PDU Router)	PDU 계층 간 라우팅 BSW 모듈
암호 서비스 관리자	CSM (Crypto Service Manager)	암호화 알고리즘 추상화 API BSW 모듈
보안 온보드 통신	SecOC (Secure On-board Communication)	CAN/Eth 메시지 MAC 인증 BSW 모듈

한국어 용어	영문 용어	설명
E2E 보호 라이브러리	E2E (End-to-End Protection Library)	통신 데이터 CRC+카운터 무결성 검증
아키텍처 XML	arxml (AUTOSAR XML)	AUTOSAR 구성 데이터 교환 표준 XML 형식
차량 신호 명세	VSS (Vehicle Signal Specification)	COVESA 차량 신호 계층적 명칭 체계
어댑티브 응용	Adaptive Application	AUTOSAR AP 위에서 동작하는 서비스/앱
기능 클러스터	Functional Cluster	AUTOSAR AP 서비스 그룹 (ara::com, ara::exec 등)
플랫폼 상태 관리	PHM (Platform Health Management)	ara::phm — 앱 생존 감시 및 복구
클라우드 네이티브 임베디드	SOAFEE	Arm 주도 차량 엣지 클라우드 네이티브 아키텍처
데이터 브로커	KUKSA.val	Eclipse 차량 신호 데이터 브로커 (VSS 기반)

18. AUTOSAR AP — ara::com 서비스 통신 심화

AUTOSAR Adaptive Platform의 ara::com은 서비스 지향 통신(SOA)을 C++ API로 구현한 Functional Cluster입니다. SOME/IP, DDS, 로컬 IPC를 동일 API로 추상화하여 바인딩 변경 없이 통신 프로토콜을 교체할 수 있습니다.

ara::com 핵심 요소	역할	C++ API
ServiceProxy	서비스 소비자 측 인터페이스	RadarServiceProxy::StartFindService(handler)
ServiceSkeleton	서비스 제공자 측 인터페이스	RadarServiceSkeleton::OfferService()
Event	비동기 이벤트 발행/구독	proxy.BrakeEvent.Subscribe(kMaxSampleCount)
Method	RPC 메소드 호출 (동기/비동기)	auto future = proxy.Calibrate(config)
Field	초기값 + 이벤트 + Get/Set	proxy.UpdateRate.Set(10) / proxy.UpdateRate.Get()
FindService	서비스 검색 (동적 바인딩)	ara::com::FindService(instance)

통신 바인딩 선택: SOME/IP(이더넷, ECU 간), DDS(도메인 내 고속 데이터), 로컬 IPC(동일 프로세스/머신). 바인딩은 aracom binding manifest(JSON/ARXML)로 배포

시 결정되어 응용 코드 변경 없이 프로토콜을 교체할 수 있습니다.

19. AUTOSAR AP 실행 관리 (ara::exec) & 상태 관리

ara::exec(Execution Management)은 Adaptive Application의 수명주기를 관리합니다. 시스템 상태(FunctionGroup)에 따라 프로세스를 시작/종료하고, PHM(Platform Health Management)이 앱 생존을 감시합니다.

AP 수명주기 상태	설명	전이 조건
Terminated	프로세스 존재하지 않음	초기 상태 또는 종료 완료
Initializing	프로세스 시작, 초기화 진행 중	EM이 프로세스 실행
Running	앱이 정상 동작 중	ReportApplicationState(kRunning) 호출 후
ShuttingDown	종료 시퀀스 진행 중	EM이 SIGTERM 전송

SM (State Management) 개념	설명	구현
FunctionGroup	논리적 기능 그룹 (예: "Driving", "Charging")	SM이 FunctionGroup 상태 전환
StateClient	앱이 SM과 통신하는 인터페이스	ara::sm::StateClient::SetState()

SM (State Management) 개념	설명	구현
FunctionGroupState	FunctionGroup 내 가능한 상태 (예: "Off", "On")	SM manifest에 정의
PHM Supervision	앱 Alive/Deadline/LogicalSu pervisory 감시	ara::phm::SupervisedEntity::R eportCheckpoint()

20. AUTOSAR CP → AP 마이그레이션 전략

기존 AUTOSAR CP 기반 ECU를 AP로 전환하는 것은 자동차 업계의 주요 과제입니다. CP와 AP는 공존(Coexistence) 아키텍처로 단계적 전환이 가능합니다.

마이그레이션 접근법	설명	적합 케이스
완전 전환 (Full Migration)	CP ECU를 AP ECU로 완전 교체	새 차종, 플랫폼 전환 기회
공존 아키텍처 (Co-existence)	CP ECU + AP HPC 병행, SOME/IP로 통신	현재 주류 — 기존 CP 재사용
계층 가상화 (Virtualization)	AP 위의 VM에서 CP 에뮬레이션	레거시 ECU 가상화, 장기 전략
기능 이전 (Function Migration)	안전 임계 기능은 CP 유지, 새 기능만 AP	안전 인증 재취득 최소화

CP→AP 마이그레이션 체크리스트: ① SOME/IP 인터페이스로 CP SWC를 서비스화 ② AP 서비스 명세(ARXML → serviceInterface manifest) 작성 ③ AP 측 Proxy/Skeleton 코드 생성 ④ CP-AP 간 게이트웨이 ECU에서 PDU↔SOME/IP 변환 설정 ⑤ 기능안전: CP는 ISO 26262, AP는 SOTIF까지 통합 분석.

21. AUTOSAR Crypto 스택 — ara::crypto

ara::crypto는 AUTOSAR AP의 암호화 서비스 Functional Cluster입니다. 알고리즘 독립적 API로 AES, RSA, ECDSA, 해시 함수를 추상화하고, HSM(Hardware Security Module)을 활용합니다.

ara::crypto 서비스	기능	알고리즘 예시
SymmetricCipher	대칭키 암호화/복호화	AES-128-GCM, AES-256-CBC
HashDigest	해시 계산	SHA-256, SHA-3-256
MessageAuthnCode	MAC 생성/검증	CMAC-AES-128, HMAC-SHA-256
DigitalSignature	서명 생성/검증	ECDSA-P256, RSA-PSS-2048
KeyEncapsulation	키 교환	ECDH, X25519
KeyDerivation	키 파생	HKDF-SHA-256, PKCS#5 PBKDF2
RandomNumberGen	난수 생성	CTR-DRBG (FIPS 140-3)

22. 혼합 임계 시스템 (Mixed Criticality System)

혼합 임계 시스템(MCS)은 ASIL D 안전 임계 기능과 QM 비임계 기능을 동일 하드웨어에서 격리하여 실행합니다. AUTOSAR AP + Hypervisor 조합이 대표 구현 방식입니다.

격리 수준	기술	안전 요구	예시
OS 파티션	AUTOSAR OS Memory Protection (MPU)	ASIL B	CP ECU 내 파티션 분리
가상화 (Type 1 Hypervisor)	QNX + Linux 동시 실행	ASIL D (QNX) + QM (Linux)	TTTech MotionWise, Green Hills INTEGRITY
컨테이너 격리	Docker + SOAFEE (OCI)	QM~ASIL B	비안전 차량 앱 격리
물리적 분리	별도 MCU/SoC	ASIL D	안전 MCU + AP HPC 이중 구조

혼합 임계 설계 원칙: ① 높은 ASIL 파티션이 낮은 파티션에 의존하면 안 됨 ② 메모리·시간 분리 증명 필요 (Freedom from Interference) ③ ASIL D 파티션의 타이밍은 Hypervisor의 시간 파티셔닝으로 보장 ④ 공유 자원(DRAM, 캐시) 간섭 분석 및 WCET 측정 필수.

23. AUTOSAR 개발 체크리스트 — 프로젝트 완수 기준

AUTOSAR 프로젝트에서 ECU 소프트웨어 인도 전 반드시 완료해야 하는 체크리스트입니다. OEM에 따라 항목이 추가되지만 아래가 최소 요건입니다.

학습 체크리스트

아래 항목을 모두 이해하고 설명할 수 있으면 다음 단계로 진행하세요.

arxml 파일 버전 일치: 개발 도구와 생성 코드의 AUTOSAR Schema 버전 동일

RTE API 컴파일 에러 없음: 전체 AUTOSAR 스택 빌드 성공

MISRA-C:2012 준수: Mandatory 규칙 위반 0건, Advisory 위반 모두 검토

E2E Profile 설정 검증: 안전 임계 PDU에 E2E 적용 및 송수신 테스트 통과

SecOC 설정 검증: 보안 임계 메시지 MAC 생성·검증 확인

OS 타이밍 검증: WCET 측정, 스택 오버플로 없음

BSW 설정 검토: NvM/DEM/DCM 설정 OEM 스펙 준수

단위 테스트 완료: MC/DC 커버리지 목표치 달성

HIL 테스트 통과: 모든 요구사항 기반 테스트 케이스 합격

진단 검증: UDS 서비스 응답 시간 OEM 스펙 내 확인

24. AUTOSAR AP — 퍼시스턴스 & 로그·트레이스

`ara::per(Persistence)`는 AP 앱이 파일 시스템 또는 키-값 저장소에 데이터를 안전하게 영속 저장하는 Functional Cluster입니다. `ara::log`와 `ara::log::LogStream`은 ECU 로그를 DLT(Diagnostic Log and Trace) 프로토콜로 출력합니다.

ara:: FC	역할	C++ API 예시
<code>ara::per::KeyValueStorage</code>	키-값 쌍으로 설정값 저장/읽기	<code>kvs.SetValue("speed_limit", 130u)</code>
<code>ara::per::FileStorage</code>	파일 기반 대용량 데이터 저장	<code>fs.OpenFileReadWrite("log.bin")</code>
<code>ara::log::Logger</code>	로그 출력 (DLT 포맷)	<code>logger.LogInfo() << "Speed:" << speed</code>
<code>ara::log::LogStream</code>	로그 스트림 빌더	<code>LogInfo() << "Init OK"</code>
DLT (AUTOSAR)	DLT 데몬으로 로그 수집·전달	Wireshark DLT 플러그인으로 분석

DLT(Diagnostic Log and Trace)는 AUTOSAR에서 채택한 로그 전송 프로토콜로, UDP 멀티캐스트나 TCP로 로그를 PC에 스트리밍합니다. DLT Viewer(무료 오픈소스)로 실시간 로그 확인 및 필터링이 가능합니다. `ara::log` 초기화:

```
ara::log::InitLogging("APP1", "앱 설명", ara::log::LogLevel::kVerbose,
ara::log::LogMode::kRemote, "").
```


25. AUTOSAR 기반 자율주행 SW 스택 — CARIAD & NVIDIA 사례

최신 자율주행 차량은 AUTOSAR CP(안전 임계)+AP(비안전 임계)+Linux(AI 처리) 3계층 혼합 구조를 사용합니다. Volkswagen 그룹의 CARIAD, 현대·기아의 MotionAI가 이 구조를 채택합니다.

계층	기능	SW 스택	하드웨어
Safety MCU (ASIL D)	EPB, ABS, EPS 안전 제어	AUTOSAR CP + ISO 26262	Infineon TC397, NXP S32K3
ADAS AP (ASIL B)	ADAS 협조 제어, SOME/IP 서비스	AUTOSAR AP + ara::com	Renesas R-Car V4H
AI 인지 HPC	딥러닝 인지, BEV, 예측	Linux + CUDA + TensorRT	NVIDIA DRIVE Orin
인포테인먼트	HMI, 내비게이션, 멀티미디어	Android Auto / AGL + QNX	Qualcomm SA8295
클라우드 연결	OTA, 데이터 수집, 원격 진단	Linux + AWS IoT, KUKSA.val	TCU (LTE/5G)

26. AUTOSAR 실무 면접 & 인증 대비 문제

AUTOSAR 실무 역량을 검증하는 면접 질문과 모범 답변 핵심 키워드입니다.

Elektrobit, Vector, ETAS 등 AUTOSAR 전문 회사 면접에서 자주 출제됩니다.

면접 질문	모범 답변 핵심 키워드
RTE 역할을 설명하시오	SWC 통신 추상화, SR/CS 포트, 이식성 보장, 생성된 코드(Rte.c/h)
AUTOSAR CP와 AP의 차이는?	CP: OSEK OS, 정적, MCU / AP: POSIX, 동적, SoC, SOME/IP, C++14
E2E 보호 적용 이유는?	데이터 무결성(CRC+카운터+DataID), 통신 결함 탐지, ASIL D 요구
SecOC와 E2E의 차이는?	E2E: 전송 오류 탐지 / SecOC: 메시지 인증(MAC), 악의적 변조 방지
arxml의 역할을 설명하시오	AUTOSAR 설계 정보 교환 XML, SWC/Interface/BSW 설정 포함
AUTOSAR OS Task 우선순위 역전 방지?	Priority Ceiling Protocol, Resource 사용(GetResource/ReleaseResource)
SWC 재사용성을 높이는 방법은?	포트 인터페이스 표준화, arxml 기반 설계, 하드웨어 독립 코드

27. AUTOSAR CP 운영 체제 — OSEK/VDX 심화

AUTOSAR CP의 OS는 OSEK/VDX(Open Systems and Corresponding Interfaces for Automotive Electronics) 표준을 기반으로 합니다. 실시간 예측 가능한 스케줄링과 최소 풋프린트가 핵심입니다.

OSEK OS 개념	설명	AUTOSAR 연관
Task (태스크)	독립 실행 단위, 선점형/협력형	OsTask — arxml로 정의
ISR (인터럽트 서비스 루틴)	ISR 카테고리 1(OS 진입 없음), 카테고리 2(OS 진입)	OsIsr — 인터럽트 우선순위 설정
Resource (리소스)	공유 자원 보호, Priority Ceiling Protocol	OsResource — 임계 영역 보호
Event	태스크 간 동기화 메커니즘	OsEvent — WaitEvent/SetEvent API
Alarm (알람)	주기적 태스크 활성화, 타이머 기반	OsAlarm — SetRelAlarm(Task, Cycle, Offset)
Counter (카운터)	알람과 스케줄 테이블의 시간 기준	OsCounter — 하드웨어 타이머 연결
Schedule Table	복잡한 주기 패턴 정의	OsScheduleTable — 다중 태스크 시간 정밀 제어

OS 설정 예 (OIL 또는 arxml): Task TASK_10ms { PRIORITY = 5; SCHEDULE = FULL; ACTIVATION = 1; }. AUTOSAR에서는 OIL 대신 arxml의 OsTask 요소로 동일 설정을 표현합니다. Stack Usage 분석: 각 태스크의 최대 스택 사용량을 정적 분석(HighTec StackAnalyzer) 또는 런타임 패턴(0xAA 마킹)으로 측정하고, OS 설정의 STACK_SIZE보다 25% 이상 여유를 두어야 합니다.

28. AUTOSAR 기반 차량 개발 전체 워크플로

AUTOSAR 기반 ECU 개발은 시스템 설계 → arxml 작성 → 코드 생성 → BSW 설정 → 통합 → 검증의 순서로 진행됩니다. 각 단계에서 사용하는 도구와 산출물을 이해하면 협업 효율이 크게 향상됩니다.

단계	활동	도구	주요 산출물
1. 시스템 설계	기능 분해, SWC 식별, 포트 정의	EA, IBM DOORS, PTC Integrity	SWC 기술서, 통신 매트릭스
2. arxml 편집	SWC/Composition/Interface 정의	Vector DaVinci Dev, EB tresos	설계 arxml 파일
3. ECU 추출	SWC를 특정 ECU에 매핑 (Mapping)	DaVinci Configurator	ECU 추출 arxml
4. BSW 설정	Com, Dem, Fim, NvM 등 BSW 파라미터	EB tresos Studio, Vector DaVinci	BSW 설정 arxml
5. RTE 생성	포트-BSW 연결 RTE 코드 자동 생성	DaVinci RTE Gen, ARCCORE Arctic	Rte.c / Rte.h
6. SWC 구현	Runnable 내 알고리즘 코딩	IDE (Eclipse, VS Code)	SWC C 소스 파일
7. 통합 빌드	SWC + BSW + OS + RTE 빌드	CMake, GHS MULTI, IAR	ECU 바이너리 (.elf/.hex)

단계	활동	도구	주요 산출물
8. 검증	SIL/HIL 테스트, CAPL 자동화	dSPACE HIL, CANoe	테스트 보고서, ASIL 증거